SAVEETHA
INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

COMPUTER SCIENCE AND ENGINEERING

Oct – 2026

ASSIGNMENT – 1

Course Code : **CSA43**

Course Name : **INTERNET PROGRAMMING**

Name : **Kandukuri. Kavya**

Registration No : **192425283**

Max Marks**: 50**

Answer all the Questions

1	Analyze using a case study the fundamental concepts of website development, HTML5 document structure, semantic elements and the important features introduced in HTML5.	(10)	(CO1)	(BL3)
2	A Professional photogragraper creates a static web page with inline, embedded and external CSS with ingredients of CSS cascading rules, specificity and inheritance with real time web .	(10)	(CO1)	(BL3)
3	Using a Go Daddy website template as a reference, explain how CSS3 features are used to design modern web pages. Discuss backgrounds, border images, colors, shadows and text effects with suitable CSS code examples	(10)	(CO1)	(BL3)
4	Design a Contemporary web page design with implementation of CSS3 transformations, transitions and animations (marquee) with different types of transformations and demonstrate how animations can be implemented using key frames.	(10)	(CO1)	(BL3)
5	Design an architecture, features and working principles of Progressive Web Applications (PWA). Discuss the roles of service workers, web app manifests, caching strategies and push notifications in providing an enhanced user experience.	(10)	(CO1)	(BL3)

QUESTION 1

Case Study: Fundamental Concepts of Website Development, HTML5 Document Structure, Semantic Elements and Important Features Introduced in HTML5

1. Introduction to Website Development

Website development is the process of creating and maintaining websites that are accessed through the internet. A website is a collection of related web pages that contain text, images, videos, links, forms and other multimedia elements.

Websites are developed for different purposes, such as education, business, entertainment, online shopping, photography portfolios and social networking.

Website development mainly consists of two parts:

1. Frontend Development: The visible part of a website with which users interact.
2. Backend Development: The server-side part responsible for processing data, managing databases and handling application logic.

The three fundamental technologies used in frontend development are:

- HTML: Defines the structure and content of a web page.
- CSS: Controls the appearance, layout, colours and design.
- JavaScript: Adds interactivity and dynamic functionality.

2. Fundamental Concepts of Website Development

Website development involves several important concepts.

2.1 HTML (HyperText Markup Language)

HTML is the standard markup language used to create web pages. It defines elements such as headings, paragraphs, images, links, tables and forms.

Example:

```
<!DOCTYPE html>
<html>
<head>
   <title>My Website</title>
</head>
<body>
   <h1>Welcome to My Website</h1>
   <p>This is my first webpage.</p>
</body>
</html>
```

In this example, HTML creates a basic web page with a heading and paragraph.

2.2 CSS (Cascading Style Sheets)

CSS is used to style HTML elements. It controls colours, fonts, spacing, borders, backgrounds and page layouts.

Example:

```
body {
   background-color: lightblue;
   font-family: Arial;
}

h1 {
   color: darkblue;
   text-align: center;
}
```

CSS improves the appearance and readability of a website.

2.3 JavaScript

JavaScript is a programming language used to make websites interactive. It can respond to user actions, validate forms, display messages and update page content.

Example:

```
<button onclick="showMessage()">Click Me</button>
<script>
function showMessage() {
   alert("Welcome to my website!");
}
</script>
```

When the user clicks the button, a message is displayed.

2.4 Web Browser

A web browser is software used to access and display websites. Examples include Google Chrome, Mozilla Firefox, Microsoft Edge and Safari.

The browser reads HTML, CSS and JavaScript files and displays the website to the user.

2.5 Web Server

A web server stores website files and sends them to users when they request a web page. Examples include Apache, Nginx and Microsoft IIS.

2.6 Domain Name and Web Hosting

A domain name is the address used to access a website, such as www.example.com.

Web hosting is a service that stores website files on a server and makes them available through the internet.

3. Types of Websites

Websites can be classified into different types based on their purpose and functionality.

3.1 Static Websites

A static website displays fixed content to users. The content usually changes only when the developer manually updates the HTML or CSS files.

Features:

- Simple structure.
- Fast loading.
- Easy to develop.
- Suitable for portfolios and informational websites.

3.2 Dynamic Websites

A dynamic website displays content that can change based on user input, database information or other conditions.

Features:

- Interactive user experience.
- Database connectivity.
- User login and registration.
- Frequently updated content.

Examples include shopping websites, social media platforms and online banking websites.

3.3 Responsive Websites

A responsive website automatically adjusts its layout according to the screen size of the device.

It works on:

- Desktop computers.
- Laptops.
- Tablets.
- Smartphones.

CSS media queries, Flexbox and Grid are commonly used to create responsive websites.

4. Introduction to HTML5

HTML5 is the fifth major version of HyperText Markup Language. It introduced several improvements to make website development easier and more powerful.

HTML5 provides better support for multimedia, semantic elements, forms, graphics and web applications.

Important features of HTML5 include:

1. Semantic elements.
2. Audio and video support.
3. Canvas graphics.

4. SVG graphics.
5. Improved forms.
6. Local storage.
7. Geolocation support.
8. Drag-and-drop functionality.
9. Responsive design support through modern web technologies.

5. Basic HTML5 Document Structure

An HTML5 document follows a standard structure. This structure helps browsers understand and display the web page correctly.

Example:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
      content="width=device-width, initial-scale=1.0">
  <title>My HTML5 Website</title>
</head>
<body>
  <header>
    <h1>Welcome to My Website</h1>
  </header>
  <nav>
    <a href="#">Home</a>
    <a href="#">About</a>
    <a href="#">Contact</a>
  </nav>
  <main>
    <section>
      <h2>About Us</h2>
      <p>This is an HTML5 webpage.</p>
    </section>
  </main>
  <footer>
    <p>Copyright 2026</p>
  </footer>
```

</body>

</html>

Explanation of HTML5 structure:

Element	Purpose
<!DOCTYPE html>	Declares the document as HTML5.
<html>	Root element of the HTML document.
<head>	Contains metadata and page information.
<meta charset>	Defines character encoding.
<meta name="viewport">	Helps the page adapt to different screen sizes.
<title>	Defines the browser tab title.
<body>	Contains visible webpage content.
<header>	Defines the introductory section.
<nav>	Contains navigation links.
<main>	Contains the main page content.
<section>	Groups related content.
<footer>	Contains footer information.

6. HTML5 Semantic Elements

Semantic elements are HTML elements that clearly describe the meaning and purpose of their content.

Before HTML5, developers commonly used elements to create different sections of a web page. HTML5 introduced semantic elements to make the document structure more meaningful.

Important semantic elements include:

6.1 <header>

The <header> element represents introductory content or a group of navigation links. It usually contains a website logo, title or heading.

```
<header>
   <h1>Photography World</h1>
   <p>Capturing Beautiful Moments</p>
</header>
```

6.2 <nav>

The <nav> element defines a section containing navigation links.

```
<nav>
   <a href="home.html">Home</a>
   <a href="about.html">About</a>
   <a href="gallery.html">Gallery</a>
</nav>
```

6.3 <main>

The <main> element represents the main content of a web page. It should contain the primary content that is unique to that page.

```
<main>
   <h2>Our Photography Services</h2>
   <p>We provide professional photography services.</p>
</main>
```

6.4 <section>

The <section> element groups related content under a common topic.

```
<section>
   <h2>Wedding Photography</h2>
   <p>We capture memorable wedding moments.</p>
</section>
```

6.5 <article>

The <article> element represents independent and self-contained content, such as a blog post, news article or product review.

```
<article>
   <h2>Photography Tips</h2>
   <p>Use natural lighting for better photographs.</p>
</article>
```

6.6 <aside>

The <aside> element contains additional information related to the main content, such as advertisements, related links or sidebars.

```
<aside>
   <h3>Related Articles</h3>
   <p>Learn more about camera settings.</p>
</aside>
```

6.7 <footer>

The <footer> element represents the footer of a page or section. It may contain copyright details, contact information and links.

```
<footer>
   <p>Copyright © 2026 Photography World</p>
</footer>
```

6.8 <figure> and <figcaption>

The <figure> element represents self-contained content such as images, diagrams or illustrations. The <figcaption> element provides a caption.

```
<figure>
   <img src="photo.jpg" alt="Mountain scenery">
   <figcaption>Beautiful mountain view</figcaption>
</figure>
```

7. Difference Between Semantic and Non-Semantic Elements

Semantic Elements	Non-Semantic Elements
Clearly describe their meaning.	Do not describe their purpose clearly.
Improve document structure.	Mainly used as general containers.
Help search engines understand content.	Provide less structural information.
Examples: <header>, <nav>, <article>.	Examples: , .

Semantic elements improve accessibility and make HTML code easier to understand and maintain.

8. Important Features Introduced in HTML5

8.1 Audio Support

HTML5 introduced the <audio> element to play sound files without requiring external plugins.

```
<audio controls>
   <source src="music.mp3" type="audio/mpeg">
   Your browser does not support audio.
</audio>
```

The controls attribute displays play, pause and volume controls.

8.2 Video Support

The <video> element allows developers to embed videos directly into web pages.

```
<video width="400" controls>
   <source src="video.mp4" type="video/mp4">
   Your browser does not support video.
</video>
```

It supports controls such as play, pause and volume.

8.3 Canvas Element

The <canvas> element provides an area for drawing graphics using JavaScript.

```
<canvas id="myCanvas"
        width="300"
        height="150">
</canvas>
<script>
let c = document.getElementById("myCanvas");
let ctx = c.getContext("2d");
ctx.fillStyle = "blue";
ctx.fillRect(20, 20, 150, 80);
</script>
```

Canvas is useful for games, charts, animations and graphical applications.

8.4 SVG Graphics

SVG stands for Scalable Vector Graphics. It is used to create vector-based images that can be resized without losing quality.

```
<svg width="200" height="100">
   <circle cx="50" cy="50" r="40"
          fill="blue" />
</svg>
```

SVG is useful for icons, logos, diagrams and illustrations.

8.5 HTML5 Forms

HTML5 introduced new input types and attributes that make form creation and validation easier.

Example:

```
<form>
   Name:
   <input type="text" required>
   Email:
   <input type="email" required>
   Date:
   <input type="date">
   Phone:
   <input type="tel">
```

```
  <input type="submit" value="Submit">
</form>
```

Some important HTML5 input types are:

- email
- date
- number
- range
- url
- tel
- time
- color

Attributes such as required, placeholder, min, max and pattern provide additional validation features.

8.6 Web Storage

HTML5 provides Web Storage to store data in the user's browser.

There are two main types:

1. Local Storage: Stores data without an automatic expiration time.
2. Session Storage: Stores data only for the current browser session.

Example:

```
<script>
localStorage.setItem("username", "Kavya");
let name = localStorage.getItem("username");
document.write(name);
</script>
```

Web Storage is useful for saving preferences, temporary information and application settings.

8.7 Geolocation API

The Geolocation API allows a website to request the user's geographical location, subject to user permission.

```
<button onclick="getLocation()">Get Location</button>
<script>
function getLocation() {
  navigator.geolocation.getCurrentPosition(
    function(position) {
      alert(position.coords.latitude);
    }
  );
```

```
}
</script>
```

It can be used in maps, location-based services and travel applications.

8.8 Drag-and-Drop

HTML5 supports drag-and-drop functionality, allowing users to move elements from one location to another.

```
<div draggable="true">
   Drag this content
</div>
```

This feature is useful in file upload interfaces, interactive boards and visual editors.

9. Case Study: Development of a Photography Website

Case Study Title: Professional Photography Portfolio Website

A professional photographer wants to develop a website to display photographs, introduce photography services and allow customers to contact the photographer for bookings.

The website should be simple, attractive and responsive. It should also provide easy navigation and support multimedia content.

9.1 Objectives of the Website

The main objectives are:

1. To display professional photographs.
2. To introduce photography services.
3. To provide information about the photographer.
4. To display contact details.
5. To allow customers to submit enquiries.
6. To provide a responsive design for different devices.

9.2 Technologies Used

Technology	**Purpose**
HTML5	Creates the structure of the website.
CSS3	Provides colours, layouts and styling.
JavaScript	Adds interactive functionality.
Web Browser	Displays the website.
Web Hosting	Makes the website available online.

9.3 Website Structure

The photography website contains the following sections:

- Header: Displays the website name and logo.
- Navigation: Provides links to different pages.

- About: Introduces the photographer.
- Gallery: Displays photographs.
- Services: Describes available photography services.
- Contact: Provides contact information and an enquiry form.
- Footer: Displays copyright and social media links.

9.4 HTML5 Implementation

The following code demonstrates the basic structure of the photography website.

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
      content="width=device-width, initial-scale=1.0">
  <title>Photography Portfolio</title>
  <style>
    body {
      font-family: Arial;
      margin: 0;
      background: #f4f4f4;
    }
    header {
      background: #222;
      color: white;
      text-align: center;
      padding: 20px;
    }
    nav {
      background: #444;
      padding: 12px;
      text-align: center;
    }
    nav a {
      color: white;
      margin: 15px;
      text-decoration: none;
    }
```

```
        section {
            padding: 25px;
            text-align: center;
        }
        img {
            width: 250px;
            margin: 10px;
        }
        footer {
            background: #222;
            color: white;
            text-align: center;
            padding: 15px;
        }
    </style>
</head>
<body>
<header>
    <h1>Photography World</h1>
    <p>Capturing Beautiful Moments</p>
</header>
<nav>
    <a href="#home">Home</a>
    <a href="#about">About</a>
    <a href="#gallery">Gallery</a>
    <a href="#contact">Contact</a>
</nav>
<main>
<section id="home">
    <h2>Welcome to Our Photography Studio</h2>
    <p>We capture your special moments.</p>
</section>
<section id="about">
    <h2>About Us</h2>
    <p>We provide professional photography services.</p>
```

```
</section>
<section id="gallery">
  <h2>Our Gallery</h2>
  <figure>
    <img src="photo1.jpg" alt="Nature photography">
    <figcaption>Nature Photography</figcaption>
  </figure>
  <figure>
    <img src="photo2.jpg" alt="Wedding photography">
    <figcaption>Wedding Photography</figcaption>
  </figure>
</section>
<section id="contact">
  <h2>Contact Us</h2>
  <form>
    <input type="text" placeholder="Your Name" required>
    <input type="email" placeholder="Your Email" required>
    <button type="submit">Submit</button>
  </form>
</section>
</main>
<footer>
  <p>Copyright © 2026 Photography World</p>
</footer>
</body>
</html>
```

9.5 Explanation of the Implementation

- The <!DOCTYPE html> declaration identifies the document as HTML5.
- The <head> section contains the page title and metadata.
- The viewport meta tag helps the page adapt to different screen sizes.
- The <header> element displays the photography studio name.
- The <nav> element provides navigation links.
- The <main> element contains the main website content.
- The <section> elements divide the page into meaningful sections.
- The <figure> and <figcaption> elements display photographs with captions.
- The <form> element allows visitors to enter their details.

10. Advantages of HTML5

HTML5 provides several advantages in modern website development.

1. Easy Structure: Semantic elements make web pages easier to organize.
2. Multimedia Support: Audio and video can be embedded without additional plugins.
3. Improved Forms: New input types support easier data collection and validation.
4. Better Accessibility: Meaningful elements help assistive technologies understand page content.
5. Graphics Support: Canvas and SVG support graphical content.
6. Offline Capabilities: Web Storage and related web technologies help applications retain data.
7. Browser Compatibility: HTML5 is supported by modern web browsers.
8. Mobile-Friendly Development: It works with responsive CSS techniques to support different devices.

11. Limitations of HTML5

Although HTML5 provides many features, it also has some limitations.

- Browser support may differ for certain features.
- HTML5 alone cannot create complete dynamic applications.
- JavaScript is required for advanced interactions.
- Client-side validation should not replace server-side validation.
- Access to features such as geolocation requires user permission.
- Multimedia files may require different formats for wider browser compatibility.

12. Applications of HTML5

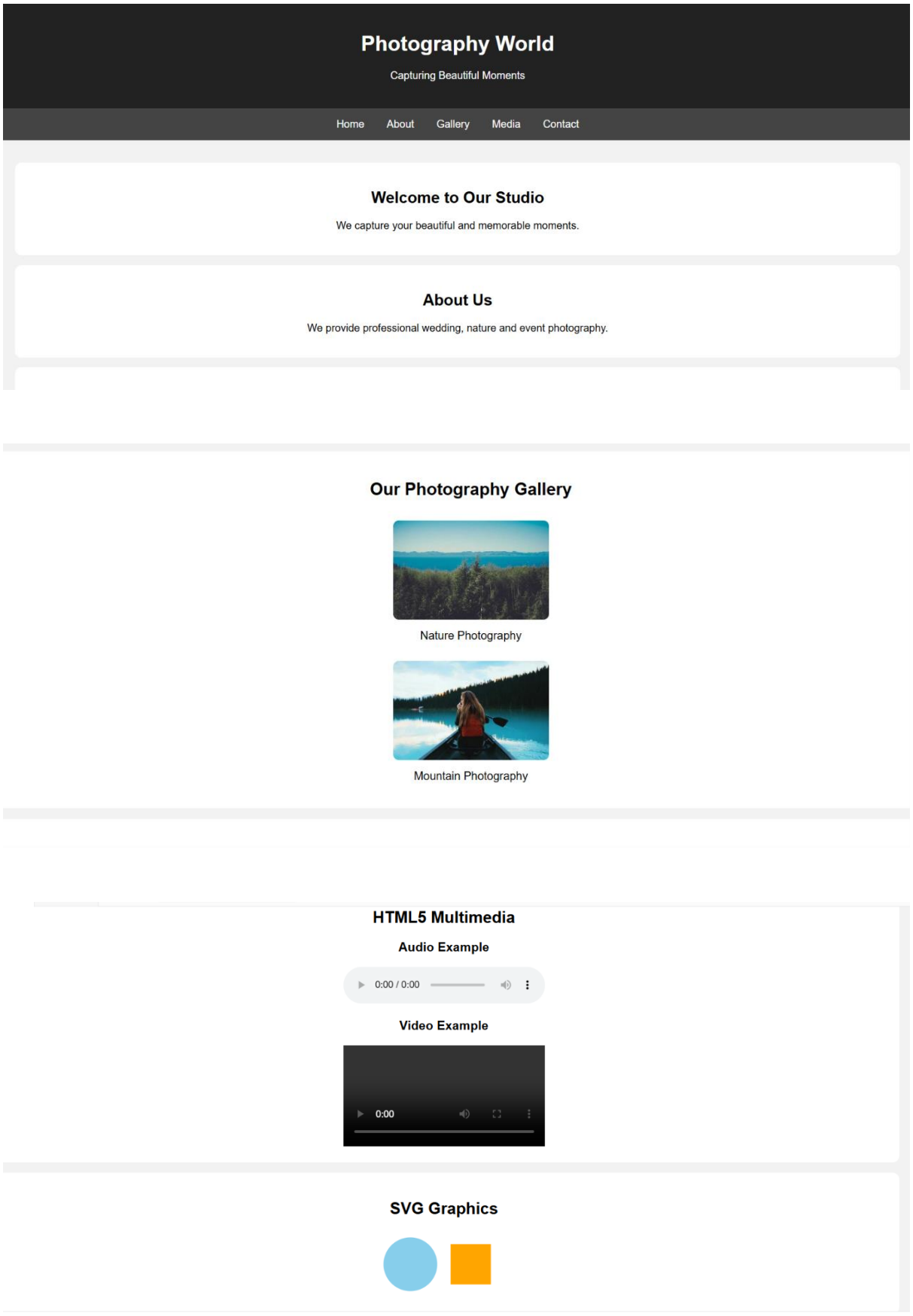
HTML5 is used in several areas of modern web development.

- Business and company websites.
- Photography and personal portfolio websites.
- Online shopping websites.
- Educational websites.
- Video and audio streaming platforms.
- Web-based games.
- Online forms and registration systems.
- Progressive Web Applications.

13. Conclusion

HTML5 plays an important role in modern website development. It provides a standard structure for creating web pages and introduces useful features such as semantic elements, audio, video, canvas, SVG, improved forms and web storage.

QUESTION 1: OUTPUT:



QUESTION 2

Case Study: Design and Development of a Professional Photographer's Static Website Using Inline, Embedded and External CSS

1. Introduction

CSS (Cascading Style Sheets) is a style sheet language used to control the appearance and layout of HTML web pages. It helps developers create attractive, organized and user-friendly websites.

A professional photographer needs a website to display photographs, introduce photography services and provide contact information to customers. The website should have an attractive design, readable text, suitable colours and an organized image gallery.

CSS helps achieve these requirements by controlling the background, font styles, colours, spacing, borders, alignment and overall layout.

In this case study, a static photography website is designed using three types of CSS:

1. Inline CSS
2. Embedded CSS
3. External CSS

The implementation also demonstrates CSS cascading rules, specificity and inheritance.

2. Objectives of the Case Study

The main objectives of developing the photography website are:

1. To create an attractive and professional photography portfolio.
2. To display photographs in an organized gallery.
3. To provide information about photography services.
4. To apply inline, embedded and external CSS.
5. To understand CSS cascading rules and specificity.
6. To demonstrate inheritance in CSS.
7. To create a simple and user-friendly website layout.

3. Introduction to Static Websites

A static website is a website in which the content remains fixed until the developer manually changes the HTML or CSS files.

It does not normally require a database or server-side programming to display its content.

Features of Static Websites:

- Simple structure and design.
- Fast loading speed.
- Easy to develop and maintain.
- Suitable for personal portfolios and business information.
- Requires fewer server resources.

Examples:

- Photography portfolio websites.
- Personal profile websites.
- Small business websites.
- Informational websites.

A photography portfolio is suitable for static website development because its main purpose is to display photographs and provide basic information.

4. Technologies Used

The photography website is developed using the following technologies.

4.1 HTML5

HTML5 is used to create the basic structure of the website. It defines headings, paragraphs, images, navigation links and other content.

4.2 CSS3

CSS3 is used to improve the appearance of the website. It controls colours, fonts, backgrounds, borders, spacing and image layouts.

4.3 Web Browser

A web browser is used to display the website. Examples include Google Chrome, Microsoft Edge and Mozilla Firefox.

4.4 Text Editor

A text editor is used to write and edit HTML and CSS code. Examples include Visual Studio Code, Notepad++ and Sublime Text.

5. Website Design and Structure

The proposed photography website contains different sections to organize the information clearly.

5.1 Header Section

The header displays the photography studio name, logo and a short introduction.

5.2 Navigation Section

The navigation bar provides links to different sections such as Home, About, Gallery and Contact.

5.3 About Section

This section introduces the photographer and provides information about their experience and photography style.

5.4 Gallery Section

The gallery displays photographs taken by the photographer. Images are arranged in an organized layout.

5.5 Services Section

This section describes the services offered, such as:

- Wedding photography.
- Nature photography.
- Event photography.
- Portrait photography.
- Product photography.

5.6 Contact Section

The contact section provides an email address, phone number or enquiry form through which customers can contact the photographer.

5.7 Footer Section

The footer contains copyright information and social media links.

6. Types of CSS

CSS can be applied to HTML documents in three different ways: inline CSS, embedded CSS and external CSS.

6.1 Inline CSS

Inline CSS is used to apply styles directly to an individual HTML element through the style attribute.

It is useful when a developer wants to apply a specific style to a single element.

Example:

```
<h1 style="color: blue; text-align: center;">
   Photography World
</h1>
```

Advantages:

- Easy to apply.
- Useful for styling individual elements.
- Does not require a separate CSS file.

Disadvantages:

- Difficult to maintain in large websites.
- Repeated styles increase code size.
- HTML and CSS become mixed together.

6.2 Embedded CSS (Internal CSS)

Embedded CSS is written inside the <style> element in the <head> section of an HTML document.

It is useful when styles are required for a single web page.

Example:

```
<head>
<style>
  body {
    background-color: lightgray;
  }
  h1 {
    color: darkblue;
    text-align: center;
  }
</style>
</head>
```

Advantages:

- Easy to manage styles for one page.
- Keeps styling rules in one location.
- No separate CSS file is required.

Disadvantages:

- Cannot be reused automatically across multiple pages.
- Increases the size of the HTML document.
- Updating several pages can become difficult.

6.3 External CSS

External CSS is written in a separate file with the .css extension. It is connected to an HTML document using the <link> element.

Example HTML:

```
<link rel="stylesheet" href="style.css">
```

Example CSS (style.css):

```
body {
  background-color: white;
  font-family: Arial;
}
h1 {
  color: purple;
  text-align: center;
}
```

Advantages:

- Separates content from presentation.
- The same CSS file can be used by multiple web pages.
- Easy to update and maintain.
- Reduces repeated styling code.

Disadvantages:

- Requires an additional CSS file.
- Incorrect file paths may prevent styles from loading.
- The browser must retrieve the external file.

7. Difference Between Inline, Embedded and External CSS

Inline CSS	**Embedded CSS**	**External CSS**
Written inside an HTML element.	Written inside the <style> tag.	Written in a separate CSS file.
Uses the style attribute.	Uses CSS selectors.	Uses CSS selectors.
Applies to an individual element.	Usually applies to one page.	Can apply to multiple pages.
Difficult to maintain for large websites.	Suitable for individual pages.	Suitable for larger websites.
Example: style="color:red"	Example: <style>h1{color:red}</style>	Example: style.css

8. CSS Cascading Rules

The term cascading refers to the process through which the browser decides which CSS rule should be applied when multiple rules target the same HTML element.

Sometimes, different CSS rules may define different values for the same property. The browser resolves these conflicts using factors such as:

1. Importance of the declaration.
2. Specificity of the selector.
3. Source order of the rules.

For example:

```
h1 {
   color: blue;
}

h1 {
   color: red;
}
```

In this example, both rules have the same specificity. Since the red rule appears later, the heading will appear in red.

Importance of CSS Cascading:

- Resolves conflicts between CSS rules.
- Helps control the final appearance of elements.
- Makes styling more organized.
- Allows developers to combine different stylesheets.

9. CSS Specificity

Specificity is the method used by browsers to determine which CSS selector has higher priority when multiple selectors target the same element.

The main specificity levels are:

Selector Type	Example	Priority
Inline style	style="color:red"	Highest among these examples
ID selector	#heading	High
Class selector	.title	Medium
Element selector	h1	Low

For example:

```
h1 {
   color: blue;
}
.title {
   color: green;
}
#main {
   color: red;
}
```

HTML:

```
<h1 class="title" id="main">Photography World</h1>
```

The heading appears in red because the ID selector has higher specificity than the class and element selectors.

Importance of Specificity:

- Helps control conflicting styles.
- Prevents unexpected design changes.
- Makes CSS rules easier to manage.
- Helps developers understand why a particular style is applied.
-

10. CSS Inheritance

Inheritance is a CSS concept in which some properties applied to a parent element are automatically passed to its child elements.

For example:

```
body {
   color: blue;
   font-family: Arial;
}
```

The text inside many child elements will inherit the blue colour and Arial font unless another rule overrides them.

Properties commonly inherited include:

- color
- font-family
- font-size
- line-height
- text-align

Properties generally not inherited include:

- margin
- padding
- border
- background
- width
- height

Advantages of Inheritance:

1. Reduces repeated CSS code.
2. Maintains consistent font styles.
3. Improves code readability.
4. Makes website styling easier to manage.

11. Real-Time Application of CSS in the Photography Website

In the photography website, CSS is applied to different elements to create a professional appearance.

The following examples explain how CSS concepts are used in the website.

11.1 Styling the Header

The header uses a dark background with white text to highlight the photography studio name.

11.2 Styling the Navigation Bar

CSS is used to arrange navigation links horizontally and provide suitable spacing between them.

11.3 Styling the Image Gallery

Images are given fixed dimensions, borders and rounded corners to create an organized gallery.

11.4 Styling the Services Section

The services section uses background colours, headings and spacing to present information clearly.

11.5 Styling the Contact Form

CSS improves the appearance of input fields and buttons to make the contact form easier to use.

12. Importance of CSS in Website Development

CSS plays an important role in creating modern websites.

1. Improved Appearance: Adds colours, fonts, borders and backgrounds.
2. Better Layout: Organizes content into suitable sections.
3. Consistency: Maintains a common design across multiple pages.
4. Responsiveness: Helps websites adapt to different screen sizes.
5. Easy Maintenance: External CSS makes design updates easier.
6. User Experience: Improves readability and navigation.
7. Reduced Repetition: Reusable styles reduce unnecessary code.

13. Advantages of Using Different CSS Methods Together

Using inline, embedded and external CSS together helps developers understand different styling approaches.

- External CSS is useful for common website styles.
- Embedded CSS is useful for page-specific styling.
- Inline CSS is useful for individual style adjustments.

However, external CSS is generally preferred for maintaining larger websites because it keeps the design separate from the HTML structure.

14. Limitations and Challenges

Some challenges that may occur while developing the photography website include:

- Conflicts between different CSS rules.
- Incorrect specificity calculations.
- Repeated styling code.
- Difficulty maintaining inline CSS.
- External stylesheet loading errors.
- Different screen sizes requiring layout adjustments.

These problems can be reduced by writing organized CSS, using meaningful class names and maintaining a consistent stylesheet.

15. Testing and Evaluation

After developing the photography website, it should be tested to ensure that all sections work correctly.

The following checks can be performed:

Test	Expected Result
Open the website	Website should load correctly.
Check header	Studio name should be displayed.
Click navigation links	Correct sections should open.
View gallery	Photographs should appear properly.
Check CSS styles	Colours and layouts should be applied.
Test contact form	Input fields should accept user details.
Resize browser	Layout should adjust appropriately.

16. Conclusion

CSS is an essential technology in modern website development because it improves the appearance, layout and usability of web pages.

In this case study, a professional photography website is designed using inline, embedded and external CSS. The concepts of cascading rules, specificity and inheritance explain how browsers apply styles to HTML elements.

External CSS provides better maintainability, embedded CSS is suitable for page-specific styling, and inline CSS is useful for individual elements.

By combining HTML5 and CSS3, developers can create attractive, organized and user-friendly static websites.

QUESTION 2: CODE:

```
<!DOCTYPE html>
<html lang="en">

<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Lens Studio</title>

    <!-- Embedded CSS -->
    <style>
        body {
            font-family: Arial, sans-serif;
            margin: 0;
            background: #f5f0eb;
            color: #333;
        }

        header {
            background: #292421;
            color: white;
            text-align: center;
            padding: 35px;
        }

        nav {
            background: #b58b68;
            padding: 15px;
            text-align: center;
        }

        nav a {
            color: white;
            text-decoration: none;
```

```
    margin: 15px;
    font-weight: bold;
}

section {
    padding: 25px;
    margin: 20px;
    background: white;
    text-align: center;
    border-radius: 12px;
}

.gallery img {
    width: 220px;
    height: 150px;
    object-fit: cover;
    margin: 10px;
    border-radius: 10px;
}

.service {
    display: inline-block;
    background: #f5e6d8;
    padding: 15px;
    margin: 10px;
    border-radius: 8px;
}

footer {
    background: #292421;
    color: white;
    text-align: center;
    padding: 15px;
}

button {
```

```
            background: #b58b68;
            color: white;
            padding: 12px 25px;
            border: none;
            border-radius: 5px;
        }

        /* CSS Specificity */
        h2 {
            color: brown;
        }

        .special {
            color: green;
        }

        #highlight {
            color: purple;
        }
    </style>
</head>

<body>

    <!-- Header with Inline CSS -->
    <header style="border-bottom: 5px solid #b58b68;">
        <h1> LENS STUDIO </h1>
        <p>Every Picture Tells a Story</p>
    </header>

    <!-- Navigation -->
    <nav>
        <a href="#home">Home</a>
        <a href="#about">About</a>
        <a href="#gallery">Gallery</a>
        <a href="#services">Services</a>
```

```
    <a href="#contact">Contact</a>
</nav>

<main>

    <!-- Home -->
    <section id="home">
        <h2 style="color: #b58b68;">
            Welcome to Lens Studio
        </h2>

        <p>Professional photography for your special moments.</p>

        <img src="https://picsum.photos/id/1015/500/250"
             width="300" alt="Beautiful landscape">
    </section>

    <!-- About -->
    <section id="about">
        <h2>About Us</h2>

        <p>
            Lens Studio is a professional photography studio
            capturing weddings, nature and special events.
        </p>
    </section>

    <!-- Gallery -->
    <section id="gallery" class="gallery">
        <h2>Our Gallery</h2>

        <img src="https://picsum.photos/id/10/400/250"
             alt="Nature photography">

        <img src="https://picsum.photos/id/1011/400/250"
             alt="Mountain photography">
```

```
    <img src="https://picsum.photos/id/1016/400/250"
        alt="Landscape photography">
</section>

<!-- Services -->
<section id="services">
    <h2>Our Services</h2>

    <div class="service">Wedding Photography</div>
    <div class="service">Nature Photography</div>
    <div class="service">Event Photography</div>
    <div class="service">Portrait Photography</div>
</section>

<!-- Specificity -->
<section>
    <h2>CSS Specificity</h2>

    <p class="special" id="highlight">
        This text demonstrates CSS specificity.
    </p>
</section>

<!-- Contact -->
<section id="contact">
    <h2>Contact Us</h2>

    <form onsubmit="submitForm(event)">

        <input type="text"
               placeholder="Your Name" required>
        <br><br>

        <input type="email"
               placeholder="Your Email" required>
```

```
            <br><br>

            <button type="submit">Send Message</button>

        </form>
    </section>

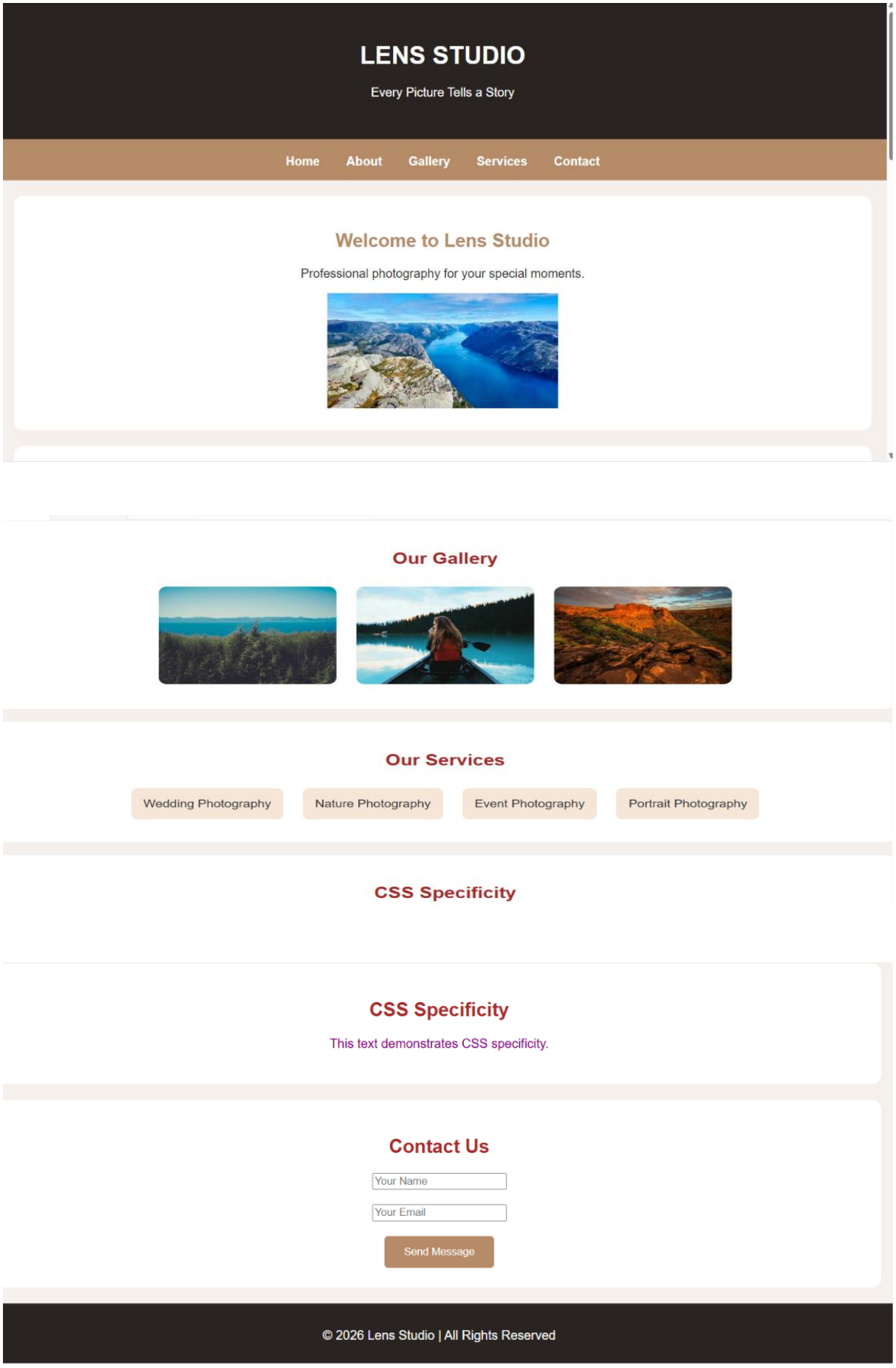
</main>

<footer>
    <p>© 2026 Lens Studio | All Rights Reserved</p>
</footer>

<script>
    function submitForm(event) {
        event.preventDefault();
        alert("Thank you for contacting Lens Studio!");
    }
</script>

</body>
</html>
```

QUESTION 2: OUTPUT:



QUESTION 3

Case Study: Designing Modern Web Pages Using CSS3 Features with Reference to GoDaddy Website Templates

1. Introduction

CSS3 (Cascading Style Sheets Level 3) is an advanced styling technology used to improve the appearance and layout of modern websites. It provides various properties that allow developers to create attractive, responsive and user-friendly web pages.

GoDaddy is a popular website-building and web-hosting platform that provides ready-made templates for businesses, portfolios, online stores and personal websites.

Modern website templates use CSS3 features such as background images, gradients, border styles, colours, shadows and text effects to create professional designs.

In this case study, a modern business website is designed using a GoDaddy-inspired template as a reference. Different CSS3 properties are explained with suitable examples to understand their practical applications.

2. Objectives of the Case Study

The main objectives are:

1. To understand the importance of CSS3 in modern website development.
2. To study the design approach used in website-builder templates.
3. To understand different CSS3 background properties.
4. To demonstrate border images and decorative borders.
5. To apply different colour formats.
6. To implement box shadows and text shadows.
7. To understand CSS text effects.
8. To develop an attractive and responsive website layout.

3. Introduction to GoDaddy Website Templates

GoDaddy provides website-building tools, hosting services and customizable website templates.

These templates allow users to create websites without writing all the HTML and CSS code manually.

GoDaddy templates are available for different purposes, including:

- Business websites.
- Photography portfolios.
- Restaurant websites.
- Online stores.
- Personal blogs.
- Professional service websites.

Important features of website templates:

- Ready-made page layouts.
- Attractive background designs.
- Pre-designed navigation menus.
- Customizable fonts and colours.
- Image galleries and content sections.
- Responsive layouts.
- Contact forms and business information.

For this case study, a generic GoDaddy-inspired business website is considered. It contains a header, navigation bar, banner, services section, image gallery and footer.

4. Introduction to CSS3

CSS3 is used to control the visual presentation of HTML elements. It allows developers to modify colours, backgrounds, borders, spacing, fonts and layouts.

CSS3 introduced several useful features that make website design easier and more flexible.

Important features of CSS3 include:

1. Background properties.
2. Border images.
3. Rounded corners.
4. Colour gradients.
5. Box shadows.
6. Text shadows.
7. Text effects.
8. Transformations.
9. Transitions.
10. Animations.
11. Media queries.

CSS3 helps developers create modern websites without depending heavily on external graphic design tools.

5. CSS3 Background Properties

Background properties are used to control the background appearance of HTML elements. They can be applied to the entire web page, header, navigation bar or individual content sections.

CSS3 supports background colours, images, gradients, positioning and image sizing.

5.1 Background Colour

The background-color property is used to set the background colour of an HTML element.

Example:

```
body {
   background-color: lightblue;
}

header {
   background-color: darkblue;
   color: white;
}
```

In this example, the entire webpage receives a light-blue background, while the header has a dark-blue background.

Application: Background colours are commonly used to distinguish different sections of a website and maintain a consistent colour theme.

5.2 Background Image

The background-image property is used to display an image behind the content of an element.

Example:

```
header {
   background-image: url("banner.jpg");
}
```

A background image can be used in the main banner of a business website to create a professional appearance.

Application: Photography websites use background images to display attractive photographs in their introductory sections.

5.3 Background Repeat

The background-repeat property controls whether a background image should repeat horizontally, vertically or in both directions.

Example:

```
body {
   background-image: url("pattern.png");
   background-repeat: no-repeat;
}
```

Common values include:

- repeat: Repeats the image in both directions.
- repeat-x: Repeats horizontally.
- repeat-y: Repeats vertically.
- no-repeat: Displays the image only once.

This property is useful when developers want to avoid unwanted image repetition.

5.4 Background Size

The background-size property controls the dimensions of a background image.

Example:

```
header {
   background-image: url("banner.jpg");
   background-size: cover;
   background-position: center;
}
```

The cover value ensures that the background image covers the entire element. The center value positions the image centrally.

Application: It is useful for creating full-width website banners and landing-page backgrounds.

5.5 CSS3 Linear Gradient

CSS3 provides gradient effects that allow two or more colours to blend smoothly.

Example:

```
header {
   background: linear-gradient(
      to right,
      black,
      brown
   );
}
```

This creates a smooth transition from black to brown.

Another example:

```
section {
   background: linear-gradient(
      lightblue,
      white
   );
}
```

Gradients improve the visual appearance of website sections without requiring separate background images.

6. CSS3 Border Images

The border-image property allows developers to use an image as the border of an HTML element.

Traditional borders use simple styles such as solid, dotted and dashed. Border images provide more decorative and creative designs.

Example:

```
.box {
   border: 15px solid transparent;
   border-image: url("border.png") 30 stretch;
}
```

Explanation:

- border specifies the border width and style.
- transparent makes the normal border colour transparent.
- border-image specifies the image used as the border.
- 30 specifies the image slicing value.
- stretch stretches the border image to fit the element.

6.1 Other Border Properties

CSS3 also provides several properties to customize borders.

Property	Purpose
border-width	Sets the thickness of the border.
border-style	Defines the appearance of the border.
border-color	Sets the border colour.
border-radius	Creates rounded corners.
border-image	Uses an image as the border.

Example:

```
.card {
   border: 2px solid brown;
   border-radius: 15px;
   padding: 20px;
}
```

This creates a card with a brown border and rounded corners.

Applications of Border Images and Borders:

- Decorative image frames.
- Photography gallery cards.
- Product display boxes.
- Special content sections.
- Promotional banners.

7. CSS3 Colour Properties

Colours play an important role in website design. They improve visual appearance, represent brand identity and help users identify important sections.

CSS3 supports different colour formats, allowing developers to select suitable colours for different website elements.

7.1 Named Colours

Named colours are predefined colour values supported by CSS.

Example:

```
h1 {
   color: blue;
}

body {
   background-color: white;
}
```

7.2 HEX Colours

HEX colours use hexadecimal values to represent different colours.

Example:

```
header {
   background-color: #333333;
}

h1 {
   color: #ffffff;
}
```

Here, #333333 represents dark grey and #ffffff represents white.

7.3 RGB Colours

RGB represents Red, Green and Blue colour values. Each value generally ranges from 0 to 255.

Example:

```
h1 {
   color: rgb(255, 0, 0);
}
```

This displays the heading in red.

7.4 RGBA Colours

RGBA includes an additional alpha value that controls transparency.

Example:

```
.box {
   background-color: rgba(0, 0, 0, 0.5);
```

}

The value 0.5 represents 50% opacity.

7.5 HSL Colours

HSL represents Hue, Saturation and Lightness.

Example:

```
h1 {
   color: hsl(210, 100%, 40%);
}
```

Importance of CSS Colours:

1. Creates an attractive website design.
2. Represents the identity of a business.
3. Improves readability.
4. Highlights important information.
5. Maintains a consistent design theme.

8. CSS3 Box Shadows

The box-shadow property is used to add shadow effects around HTML elements.

It is commonly used in modern website templates to create depth and highlight important content.

Syntax:

```
box-shadow: horizontal vertical blur spread color;
```

Example:

```
.card {
   background: white;
   box-shadow: 5px 5px 10px gray;
}
```

Explanation:

- Horizontal value controls the shadow's horizontal position.
- Vertical value controls the vertical position.
- Blur controls the softness of the shadow.
- Spread controls the size of the shadow.
- Colour defines the shadow colour.

8.1 Multiple Box Shadows

CSS3 allows multiple shadows to be applied to the same element.

Example:

```
.card {
   box-shadow:
      2px 2px 5px gray,
      0 0 10px lightblue;
}
```

This produces more than one shadow effect around the card.

8.2 Hover Shadow Effect

A shadow can be increased when the user moves the mouse over an element.

Example:

```
.card:hover {
   box-shadow: 0 8px 20px gray;
}
```

Applications of Box Shadows:

- Service cards.
- Product boxes.
- Navigation menus.
- Image galleries.
- Buttons and content panels.

Box shadows create a layered appearance and make important website elements stand out.

9. CSS3 Text Effects

CSS3 provides several properties to improve the appearance and readability of text.

These properties are useful for creating attractive headings, professional banners and clearly organized website content.

9.1 Text Shadow

The text-shadow property adds a shadow behind text.

Example:

```
h1 {
   text-shadow: 2px 2px 5px gray;
}
```

This produces a shadow behind the heading, making it visually prominent.

9.2 Text Overflow

The text-overflow property controls how hidden text is displayed when it exceeds the available space.

Example:

```
p {
```

```
  width: 200px;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}
```

The ellipsis value displays three dots when the text is cut off.

9.3 Word Wrapping

The overflow-wrap property allows long words to break and move to the next line when necessary.

Example:

```
p {
  overflow-wrap: break-word;
}
```

It helps prevent long text from extending outside its container.

9.4 Text Transformation

The text-transform property changes the capitalization of text.

Example:

```
h2 {
  text-transform: uppercase;
}
```

Common values include:

- uppercase
- lowercase
- capitalize
- none

9.5 Letter Spacing

The letter-spacing property controls the space between individual characters.

Example:

```
h1 {
  letter-spacing: 3px;
}
```

It is commonly used for website titles and headings.

9.6 Word Spacing

The word-spacing property controls the space between words.

Example:

```
p {
```

```
    word-spacing: 5px;
}
```

These text effects improve the overall presentation of website content.

10. Application of CSS3 Features in a Modern Website

A GoDaddy-inspired business website can use the above CSS3 properties to create a professional and attractive appearance.

The following sections explain how these properties are applied in a practical website.

10.1 Header and Banner

The header displays the company name and introductory information. A background image or gradient can be used to create an attractive banner.

CSS properties such as background-image, background-size, background-position and text-shadow improve its appearance.

10.2 Navigation Menu

The navigation menu contains links to different website sections.

CSS properties such as background-color, padding, color and text-decoration help create a clean navigation design.

10.3 Service Cards

Service cards display information about different business services.

The background-color, border-radius and box-shadow properties help separate each service into an individual card.

10.4 Image Gallery

The image gallery displays photographs or product images in an organized layout.

CSS properties such as border-radius, border, box-shadow and object-fit improve the presentation of images.

10.5 Buttons

Buttons are used for actions such as contacting the business, viewing services or booking appointments.

CSS properties such as background-color, border-radius, padding and box-shadow make buttons more attractive.

10.6 Footer

The footer displays copyright details, contact information and social media links.

A dark background with light-coloured text creates a clear distinction between the footer and the main content.

11. Responsive Design Using CSS3

Responsive web design is the process of creating websites that automatically adjust their layouts according to the screen size of the device.

Modern website templates are designed to work on desktop computers, laptops, tablets and smartphones.

CSS3 media queries are used to apply different styles for different screen sizes.

Example:

```
@media (max-width: 600px) {
  .container {
    width: 100%;
  }

  nav {
    display: block;
  }

  .card {
    width: 100%;
  }
}
```

Advantages of Responsive Design:

- Supports different mobile devices.
- Improves user experience.
- Reduces horizontal scrolling.
- Provides flexible layouts.
- Makes websites easier to access.

12. Advantages of CSS3 in Modern Website Design

CSS3 provides several advantages for website development.

1. Attractive Design: Provides advanced visual effects such as gradients, shadows and rounded corners.
2. Easy Styling: Allows developers to style HTML elements using simple properties.
3. Reduced Image Usage: Gradients and shadows can be created directly using CSS.
4. Responsive Layouts: Media queries support different devices and screen sizes.
5. Better User Experience: Improves readability and visual presentation.
6. Reusable Styles: Common styles can be applied across multiple web pages.
7. Easy Maintenance: CSS files can be modified without changing the HTML structure.
8. Flexible Design: Developers can customize layouts according to business requirements.

13. Limitations of CSS3

Although CSS3 provides many useful features, it also has certain limitations.

- Some advanced properties may behave differently across browsers.
- Complex CSS can become difficult to maintain.
- Excessive shadows and visual effects may reduce readability.
- Incorrect background image sizing can affect the layout.
- Responsive designs require testing on different devices.
- CSS alone cannot provide complete interactive functionality.

These limitations can be reduced by using organized stylesheets and testing websites in different browsers.

14. Testing and Evaluation

After developing a modern website using CSS3, different tests should be performed to ensure that the design works correctly.

Test	Expected Result
Background test	Background colours and images should display correctly.
Border test	Border images and rounded corners should appear properly.
Colour test	Selected colours should be applied correctly.
Shadow test	Box shadows should appear around selected elements.
Text effect test	Text shadows and transformations should work.
Responsive test	Layout should adjust to different screen sizes.
Browser test	Website should display correctly in modern browsers.

Testing helps identify design problems and improves the quality of the website.

15. Real-World Applications of CSS3

CSS3 is widely used in different types of modern websites.

1. Business Websites: Used to create professional layouts and service sections.
2. Photography Portfolios: Used to style image galleries and banners.
3. E-commerce Websites: Used to design product cards, buttons and promotional sections.
4. Educational Websites: Used to organize learning content and navigation menus.
5. Restaurant Websites: Used to display menus, images and contact information.
6. Personal Blogs: Used to improve text presentation and page layouts.
7. Company Landing Pages: Used to create attractive banners and business information sections.

Website builders such as GoDaddy use modern styling techniques to provide customizable templates for different industries.

16. Conclusion

CSS3 plays an important role in designing modern and professional websites. It provides advanced styling features such as backgrounds, border images, colours, box shadows and text effects.

In this case study, a GoDaddy-inspired website design is considered to understand how CSS3 properties are applied to different website sections.

Background properties improve the appearance of page sections, border images provide decorative designs, colours create visual identity, box shadows add depth and text effects improve the presentation of content.

By combining HTML5 and CSS3, developers can create attractive, responsive and user-friendly websites suitable for modern business requirements.

QUESTION 3: CODE:

```
<!DOCTYPE html>
<html>
<head>
   <title>Creative Studio</title>

   <style>
      * {
         margin: 0;
         padding: 0;
         box-sizing: border-box;
      }

      body {
         font-family: Arial, sans-serif;
         background-color: #f5f1eb;
         color: #333;
      }

      /* Navigation */
      header {
         background: #29221d;
         color: white;
         padding: 20px;
         text-align: center;
      }

      nav a {
         color: white;
         text-decoration: none;
         margin: 15px;
```

```
}

/* Background and gradient */
.hero {
    background: linear-gradient(120deg, #493629, #b58b67);
    color: white;
    text-align: center;
    padding: 90px 20px;
}

.hero h1 {
    font-size: 42px;
    text-shadow: 3px 3px 5px black;
    margin-bottom: 15px;
}

.hero p {
    font-size: 18px;
}

.btn {
    display: inline-block;
    margin-top: 25px;
    padding: 12px 25px;
    background: #fff;
    color: #493629;
    text-decoration: none;
    border-radius: 5px;
    box-shadow: 3px 3px 8px #222;
}

/* About section */
.about {
    text-align: center;
    padding: 40px 20px;
}
```

```
.about h2 {
    color: #805b40;
    margin-bottom: 15px;
}

/* Border image effect */
.gallery {
    padding: 40px 20px;
    text-align: center;
}

.gallery h2 {
    margin-bottom: 25px;
    color: #493629;
}

.cards {
    display: flex;
    justify-content: center;
    flex-wrap: wrap;
    gap: 25px;
}

.card {
    background: white;
    width: 260px;
    padding: 20px;
    border: 8px solid;
    border-image: linear-gradient(45deg, #805b40, #e6c9a8) 1;
    box-shadow: 4px 4px 12px rgba(0,0,0,0.2);
    transition: 0.4s;
}

.card:hover {
    transform: translateY(-8px);
```

```
    box-shadow: 6px 8px 18px rgba(0,0,0,0.3);
}

.card img {
    width: 100%;
    height: 150px;
    object-fit: cover;
    margin-bottom: 12px;
}

.card h3 {
    color: #805b40;
    margin-bottom: 10px;
}

/* Text effects */
.effects {
    background: #e7d8c8;
    padding: 40px 20px;
    text-align: center;
}

.effects h2 {
    text-transform: uppercase;
    letter-spacing: 3px;
    text-shadow: 2px 2px 3px #aaa;
    margin-bottom: 15px;
}

.effects p {
    overflow-wrap: break-word;
    line-height: 1.7;
}

/* Footer */
footer {
```

```
        background: #29221d;
        color: white;
        text-align: center;
        padding: 20px;
      }

      /* Responsive design */
      @media (max-width: 600px) {
        .hero h1 {
          font-size: 28px;
        }

        nav a {
          display: inline-block;
          margin: 8px;
        }

        .cards {
          flex-direction: column;
          align-items: center;
        }

        .card {
          width: 90%;
        }
      }
    </style>
</head>

<body>

    <header>
      <h2>CREATIVE STUDIO</h2>
      <nav>
        <a href="#home">Home</a>
        <a href="#about">About</a>
```

```
        <a href="#gallery">Services</a>
        <a href="#contact">Contact</a>
    </nav>
</header>

<section class="hero" id="home">
    <h1>Design Your Dream Website</h1>
    <p>Beautiful, modern and responsive website designs.</p>
    <a href="#gallery" class="btn">Explore More</a>
</section>

<section class="about" id="about">
    <h2>About Our Studio</h2>
    <p>
        We create attractive websites using modern HTML5
        and CSS3 technologies. Our designs are simple,
        creative and suitable for all devices.
    </p>
</section>

<section class="gallery" id="gallery">
    <h2>Our Services</h2>

    <div class="cards">

        <div class="card">
            <img src="https://picsum.photos/300/200?random=1"
                alt="Web design">
            <h3>Web Design</h3>
            <p>Creative and attractive website layouts.</p>
        </div>

        <div class="card">
            <img src="https://picsum.photos/300/200?random=2"
                alt="Photography">
            <h3>Photography</h3>
```

```
            <p>Professional photography and editing services.</p>
        </div>

        <div class="card">
            <img src="https://picsum.photos/300/200?random=3"
                alt="Development">
            <h3>Development</h3>
            <p>Responsive websites for modern businesses.</p>
        </div>

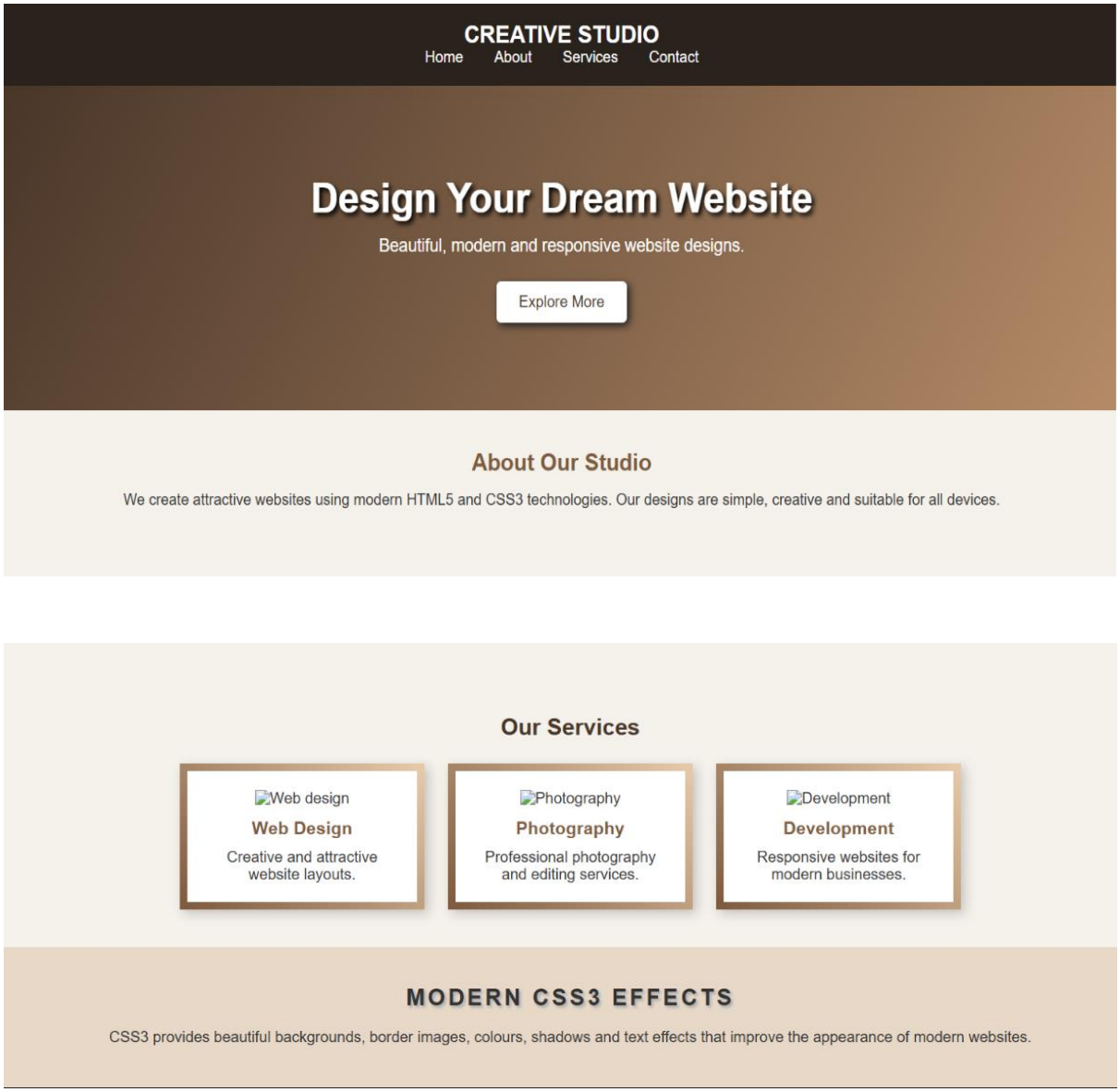
    </div>
</section>

<section class="effects">
    <h2>Modern CSS3 Effects</h2>
    <p>
        CSS3 provides beautiful backgrounds, border images,
        colours, shadows and text effects that improve
        the appearance of modern websites.
    </p>
</section>

<footer id="contact">
    <p>Contact: creativestudio@example.com</p>
    <p>&copy; 2026 Creative Studio. All Rights Reserved.</p>
</footer>

</body>
</html>
```

QUESTION 3: OUTPUT:



Question 4: Design a Contemporary Web Page Using CSS3 Transformations, Transitions and Animations

1. Introduction

CSS3 is an advanced version of Cascading Style Sheets used to improve the appearance and functionality of web pages. It provides several features such as transformations, transitions and animations that make websites more attractive, interactive and user-friendly.

Modern websites use these features to create moving elements, animated headings, image effects, buttons and interactive layouts. CSS3 reduces the need for JavaScript when implementing simple visual effects.

In this case study, a contemporary website called Creative Design Studio is designed using CSS3 transformations, transitions and animations.

2. Objectives

The main objectives of this design are:

1. To understand CSS3 transformations and their different types.
2. To implement transitions for smooth visual effects.
3. To create animations using CSS keyframes.
4. To understand the working of the marquee effect.
5. To design an attractive and responsive web page.
6. To improve user interaction through animation and movement effects.

3. Introduction to CSS3 Transformations

CSS3 transformations are used to change the position, size, angle and shape of HTML elements without affecting the normal document flow.

Transformations can be applied to images, buttons, headings, cards and other web elements.

The CSS transform property is used to apply these effects.

Syntax:

```
element {
   transform: function(value);
}
```

For example:

```
.box {
   transform: rotate(20deg);
}
```

This rotates the selected element by 20 degrees.

4. Types of CSS3 Transformations

CSS3 supports different types of transformations. The major types are translate, rotate, scale and skew.

4.1 Translate Transformation

Translate is used to move an element from its original position along the X-axis and Y-axis.

- translateX() moves an element horizontally.
- translateY() moves an element vertically.
- translate() moves an element in both directions.

Example:

```
.box {
   transform: translate(30px, 20px);
}
```

This moves the element 30 pixels to the right and 20 pixels downward.

Translate is commonly used for moving buttons, cards and images when users interact with them.

4.2 Rotate Transformation

Rotate is used to rotate an element clockwise or anticlockwise around its center or another specified point.

The rotate() function accepts an angle in degrees.

Example:

```
.image {
   transform: rotate(15deg);
}
```

This rotates the image by 15 degrees.

Rotation is useful for creating attractive image effects, icons and decorative elements.

4.3 Scale Transformation

Scale is used to increase or decrease the size of an element.

- scale(1) maintains the original size.
- scale(1.5) increases the size.
- scale(0.5) reduces the size.

Example:

```
.card:hover {
   transform: scale(1.1);
}
```

When the user moves the mouse over the card, its size increases by 10%.

Scale transformations are commonly used in image galleries, product cards and buttons.

4.4 Skew Transformation

Skew is used to tilt an element horizontally or vertically.

- skewX() tilts an element along the X-axis.

- skewY() tilts an element along the Y-axis.
- skew() tilts an element along both axes.

Example:

```
.box {
   transform: skew(15deg, 5deg);
}
```

This tilts the element by 15 degrees horizontally and 5 degrees vertically.

Skew effects are used in creative website layouts and decorative designs.

4.5 Transform Origin

The transform-origin property specifies the point around which an element is transformed.

By default, transformations take place around the center of the element.

Example:

```
.box {
   transform: rotate(30deg);
   transform-origin: top left;
}
```

This rotates the element around its top-left corner.

5. Two-Dimensional and Three-Dimensional Transformations

CSS3 transformations are classified into two main categories.

Two-dimensional transformations (2D):

2D transformations change elements along the X-axis and Y-axis. They include translation, rotation, scaling and skewing.

```
.box {
   transform: translate(20px, 10px) rotate(10deg);
}
```

Three-dimensional transformations (3D):

3D transformations add depth to web elements by using the X, Y and Z axes.

Some important 3D functions are:

- rotateX() – rotates an element around the X-axis.
- rotateY() – rotates an element around the Y-axis.
- rotateZ() – rotates an element around the Z-axis.
- translateZ() – moves an element along the Z-axis.

Example:

```
.box {
   transform: rotateY(40deg);
}
```

This creates a three-dimensional rotation effect.

6. Introduction to CSS3 Transitions

CSS3 transitions are used to change the appearance of an element smoothly over a specified period.

Without transitions, changes such as colour, size and position may happen immediately. Transitions make these changes gradual and attractive.

The transition property is used to control the effect.

Syntax:

```
element {
   transition: property duration timing-function delay;
}
```

The main transition properties are:

1. transition-property – specifies which CSS property should change.
2. transition-duration – specifies how long the transition takes.
3. transition-timing-function – controls the speed of the transition.
4. transition-delay – specifies when the transition should begin.

7. Types of Transition Timing Functions

CSS3 provides different timing functions to control the speed of transitions.

Timing function	Description
ease	Starts slowly, becomes faster and ends slowly.
linear	Maintains the same speed throughout.
ease-in	Starts slowly and becomes faster.
ease-out	Starts quickly and becomes slower.
ease-in-out	Starts and ends slowly.

Example:

```
button {
   background-color: blue;
   transition: background-color 1s ease;
}

button:hover {
   background-color: orange;
}
```

When the user moves the mouse over the button, its background colour changes smoothly from blue to orange.

8. Transition Effects on Images and Cards

Transitions can be applied to images and cards to improve the appearance of a website.

For example, an image can gradually increase in size when the mouse moves over it.

Example:

```
img {
   transition: transform 0.5s;
}

img:hover {
   transform: scale(1.2);
}
```

This creates a smooth zoom effect.

Similarly, cards can move upward when the user hovers over them.

```
.card {
   transition: transform 0.4s ease;
}

.card:hover {
   transform: translateY(-10px);
}
```

These effects are commonly used in online shopping websites, portfolios and business websites.

9. Introduction to CSS3 Animations

CSS3 animations are used to change the appearance and position of elements automatically over a period of time.

Unlike transitions, animations do not always require user interaction. They can start automatically when a web page loads.

Animations are created using the @keyframes rule.

Keyframes define the different stages of an animation and specify how an element should change during each stage.

Basic Syntax:

```
@keyframes animationname {
   0% {
      property: value;
   }

   100% {
```

```
    property: value;
  }
}
```

The animation is applied to an element using the animation property.

```
.box {
  animation: move 3s infinite;
}
```

Here, the animation runs for three seconds and repeats continuously.

10. Important Animation Properties

CSS3 provides several properties to control animations.

Property	Description
animation-name	Specifies the name of the animation.
animation-duration	Defines the time taken to complete the animation.
animation-delay	Specifies the starting delay.
animation-iteration-count	Defines how many times the animation repeats.
animation-direction	Specifies the direction of animation.
animation-timing-function	Controls the speed of animation.
animation-fill-mode	Defines the style applied before or after animation.

Example:

```
.box {
  animation-name: move;
  animation-duration: 4s;
  animation-iteration-count: infinite;
  animation-direction: alternate;
}
```

This makes the element move repeatedly in alternate directions.

11. Creating Animations Using Keyframes

The @keyframes rule allows developers to define different stages of an animation.

For example, an element can move from left to right while changing its background colour.

Example:

```
@keyframes colorchange {
  0% {
    background-color: red;
  }

  50% {
```

```
    background-color: yellow;
  }

  100% {
    background-color: blue;
  }
}

.box {
  animation: colorchange 4s infinite;
}
```

In this example:

- At 0%, the background is red.
- At 50%, the background changes to yellow.
- At 100%, the background becomes blue.

The animation repeats continuously.

12. Different Types of CSS Animations

CSS3 supports different animation effects that can improve the appearance of websites.

1. Fade Animation: Gradually changes the visibility of an element.

```
@keyframes fade {
  from { opacity: 0; }
  to { opacity: 1; }
}
```

2. Slide Animation: Moves an element from one position to another.

```
@keyframes slide {
  from { transform: translateX(-100px); }
  to { transform: translateX(0); }
}
```

3. Zoom Animation: Increases or decreases the size of an element.

```
@keyframes zoom {
  from { transform: scale(0.5); }
  to { transform: scale(1); }
}
```

4. Rotation Animation: Rotates an element continuously.

```
@keyframes rotate {
  from { transform: rotate(0deg); }
```

```
    to { transform: rotate(360deg); }
}
```

These animations can be applied to images, icons, headings and other elements.

13. Marquee Effect Using CSS3

A marquee is a scrolling text effect in which text moves horizontally or vertically across a web page.

Earlier, HTML provided the <marquee> tag for this purpose. However, it is obsolete and should not be used in modern web development.

CSS3 animations provide a better method for creating scrolling text.

Example:

```
<div class="marquee">
    Welcome to Creative Design Studio!
</div>
.marquee {
    white-space: nowrap;
    overflow: hidden;
    animation: scroll 8s linear infinite;
}

@keyframes scroll {
    from {
        transform: translateX(100%);
    }

    to {
        transform: translateX(-100%);
    }
}
```

The text continuously moves from right to left across the screen.

Marquee effects are useful for announcements, news updates, offers and notifications.

14. Case Study: Creative Design Studio Website

A Creative Design Studio wants to develop a modern website to display its services, portfolio and contact information.

The website uses CSS3 transformations, transitions and animations to improve user interaction.

The main sections of the website include:

1. Header Section: Displays the website name and navigation links.

2. Hero Section: Contains an attractive heading and animated text.
3. Services Section: Displays different services using cards.
4. Gallery Section: Shows images with zoom and rotation effects.
5. Announcement Section: Displays scrolling text using a marquee effect.
6. Footer Section: Displays contact details and copyright information.

CSS3 transformations are used to move and resize cards. Transitions provide smooth hover effects. Animations are used for moving text and decorative elements.

15. Implementation of CSS3 Effects in the Website

The following code examples demonstrate how different effects are implemented in the Creative Design Studio website.

Moving service cards:

```
.card:hover {
   transform: translateY(-10px);
   transition: 0.5s;
}
```

Zooming gallery images:

```
.gallery img {
   transition: transform 0.5s;
}

.gallery img:hover {
   transform: scale(1.1);
}
```

Rotating logo:

```
.logo:hover {
   transform: rotate(360deg);
   transition: 1s;
}
```

Animated heading:

```
@keyframes heading {
   from {
      opacity: 0;
      transform: translateY(-30px);
   }

   to {
```

```
    opacity: 1;
    transform: translateY(0);
  }
}

h1 {
  animation: heading 2s ease;
}
```

These effects make the website more attractive and interactive.

16. Responsive Design and Browser Compatibility

Responsive design ensures that a website works properly on different screen sizes, including mobile phones, tablets and desktop computers.

CSS3 media queries are used to adjust the layout according to the screen width.

Example:

```
@media (max-width: 600px) {
  .container {
    flex-direction: column;
  }

  h1 {
    font-size: 25px;
  }
}
```

This changes the layout into a vertical arrangement on smaller screens.

Developers should also test animations and transitions in different browsers such as Chrome, Firefox, Edge and Safari.

The prefers-reduced-motion media query can be used to reduce unnecessary animation for users who prefer less movement.

```
@media (prefers-reduced-motion: reduce) {
  * {
    animation: none !important;
    transition: none !important;
  }
}
```

17. Advantages of CSS3 Transformations, Transitions and Animations

1. They improve the visual appearance of websites.

2. They provide smooth and attractive effects.
3. They improve user interaction.
4. They reduce the need for JavaScript for simple animations.
5. They make websites more modern and engaging.
6. They support responsive website design.
7. They can be applied to images, buttons, text and other elements.
8. They help create professional website layouts.

18. Limitations of CSS3 Effects

Although CSS3 provides many advantages, it also has certain limitations.

1. Excessive animations can distract users.
2. Complex animations may affect website performance.
3. Some effects may behave differently across browsers.
4. Large numbers of animated elements may consume system resources.
5. Poorly designed animations can reduce readability.
6. Developers must consider accessibility for users who prefer reduced motion.

19. Testing and Evaluation

The completed website should be tested to ensure that all CSS3 effects work correctly.

Test	Expected Result
Translate	Element moves from its original position.
Rotate	Element rotates at the specified angle.
Scale	Element increases or decreases in size.
Skew	Element appears tilted.
Transition	Changes occur smoothly.
Keyframe animation	Animation runs automatically.
Marquee	Text scrolls continuously.
Responsive design	Layout adjusts to screen size.

Testing helps identify errors and ensures that the website provides a good user experience.

20. Real-World Applications

CSS3 transformations, transitions and animations are widely used in modern websites.

- E-commerce websites: Product zoom effects and interactive shopping cards.
- Portfolio websites: Animated headings and image galleries.
- Business websites: Smooth navigation and attractive buttons.
- News websites: Scrolling announcements and headlines.
- Educational websites: Animated diagrams and interactive learning elements.
- Entertainment websites: Moving banners and visual effects.

21. Conclusion

CSS3 transformations, transitions and animations play an important role in modern web development. Transformations are used to change the position, size and angle of elements. Transitions provide smooth changes between different styles, while animations create automatic movement and visual effects using keyframes.

The Creative Design Studio case study demonstrates how these features can be combined to develop an attractive, interactive and responsive website. By using CSS3 effectively, developers can improve website appearance, usability and user experience.

QUESTION 4: CODE:

```
<!DOCTYPE html>
<html>
<head>
  <title>Creative Design Studio</title>

  <style>
    * {
      margin: 0;
      padding: 0;
      box-sizing: border-box;
    }

    body {
      font-family: Arial, sans-serif;
      background: #f4f4f4;
      color: #333;
    }

    /* Header */
    header {
      background: #222;
      color: white;
      padding: 20px;
      text-align: center;
```

```
}

nav {
    margin-top: 12px;
}

nav a {
    color: white;
    text-decoration: none;
    margin: 15px;
}

/* Hero section */
.hero {
    background: linear-gradient(120deg, #493879, #9b59b6);
    color: white;
    text-align: center;
    padding: 70px 20px;
}

.hero h1 {
    font-size: 38px;
    animation: fade 2s ease;
}

.hero p {
    margin-top: 15px;
}

/* Fade animation */
@keyframes fade {
    from {
        opacity: 0;
        transform: translateY(-40px);
    }
```

```
    to {
        opacity: 1;
        transform: translateY(0);
    }
}

/* Marquee */
.marquee {
    background: #f1c40f;
    padding: 12px;
    overflow: hidden;
    white-space: nowrap;
}

.marquee p {
    display: inline-block;
    animation: scroll 10s linear infinite;
}

@keyframes scroll {
    from {
        transform: translateX(100%);
    }

    to {
        transform: translateX(-100%);
    }
}

/* Transformations */
.section {
    padding: 40px 20px;
    text-align: center;
}

.cards {
```

```
    display: flex;
    justify-content: center;
    flex-wrap: wrap;
    gap: 25px;
    margin-top: 25px;
}

.card {
    background: white;
    padding: 25px;
    width: 220px;
    border-radius: 10px;
    box-shadow: 2px 2px 10px #aaa;
    transition: 0.5s;
}

/* Translate and scale */
.card:hover {
    transform: translateY(-10px) scale(1.05);
    background: #e8d9f5;
}

/* Rotate example */
.rotate {
    display: inline-block;
    margin: 20px;
    padding: 20px;
    background: #3498db;
    color: white;
    transition: 1s;
}

.rotate:hover {
    transform: rotate(15deg);
}
```

```
/* Skew example */
.skew {
    display: inline-block;
    margin: 20px;
    padding: 20px;
    background: #e67e22;
    color: white;
    transition: 0.5s;
}

.skew:hover {
    transform: skew(15deg);
}

/* Continuous rotation */
.circle {
    width: 70px;
    height: 70px;
    background: #27ae60;
    margin: 30px auto;
    border-radius: 50%;
    animation: rotate 3s linear infinite;
}

@keyframes rotate {
    from {
        transform: rotate(0deg);
    }

    to {
        transform: rotate(360deg);
    }
}

/* Footer */
footer {
```

```
        background: #222;
        color: white;
        text-align: center;
        padding: 20px;
      }

      /* Responsive design */
      @media (max-width: 600px) {
        .hero h1 {
          font-size: 26px;
        }

        .cards {
          flex-direction: column;
          align-items: center;
        }

        nav a {
          display: inline-block;
          margin: 8px;
        }
      }
    </style>
</head>

<body>

    <header>
      <h2>CREATIVE DESIGN STUDIO</h2>

      <nav>
        <a href="#home">Home</a>
        <a href="#services">Services</a>
        <a href="#effects">Effects</a>
        <a href="#contact">Contact</a>
      </nav>
```

```
</header>

<section class="hero" id="home">
  <h1>Welcome to Creative Studio</h1>
  <p>Explore modern website designs using CSS3.</p>
</section>

<div class="marquee">
  <p>Welcome! Explore our creative designs and latest services!</p>
</div>

<section class="section" id="services">
  <h2>Our Services</h2>

  <div class="cards">

    <div class="card">
      <h3>Web Design</h3>
      <p>Creative and modern website layouts.</p>
    </div>

    <div class="card">
      <h3>Development</h3>
      <p>Responsive websites for all devices.</p>
    </div>

    <div class="card">
      <h3>Photography</h3>
      <p>Professional photography services.</p>
    </div>

  </div>
</section>

<section class="section" id="effects">
  <h2>CSS3 Transformation Effects</h2>
```

```
        <div class="rotate">
            Rotate Effect
        </div>

        <div class="skew">
            Skew Effect
        </div>

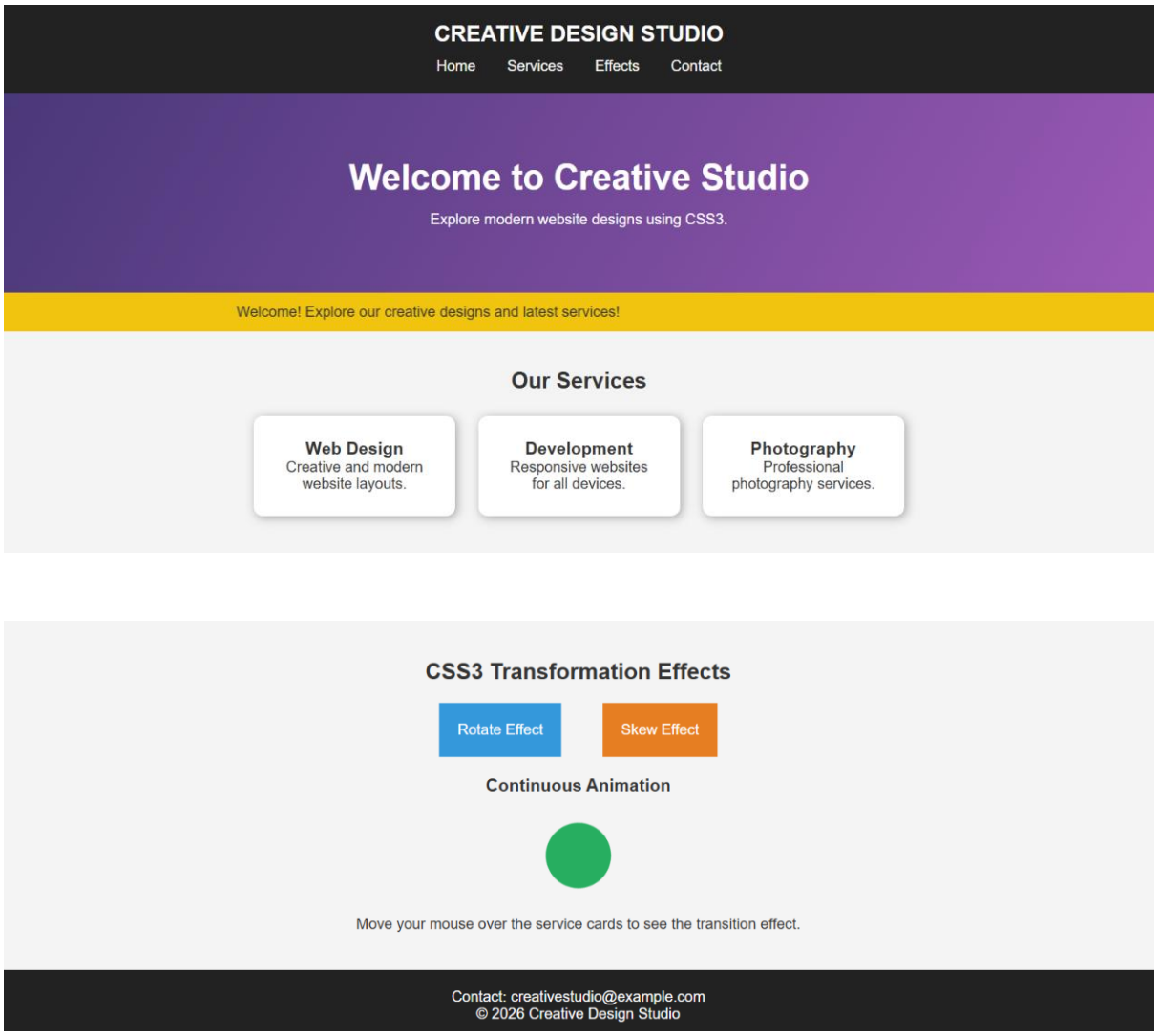
        <h3>Continuous Animation</h3>
        <div class="circle"></div>

        <p>Move your mouse over the service cards to see the transition effect.</p>
    </section>

    <footer id="contact">
        <p>Contact: creativestudio@example.com</p>
        <p>&copy; 2026 Creative Design Studio</p>
    </footer>

</body>
</html>
```

QUESTION 4: OUTPUT :



Question 5: Design the Architecture, Features and Working Principles of Progressive Web Applications (PWA)

1. Introduction

A Progressive Web Application (PWA) is a modern type of web application that combines the features of a website and a mobile application. It provides an app-like experience directly through a web browser without requiring users to download an application from an app store.

PWAs are developed using standard web technologies such as HTML, CSS and JavaScript. They can work on different devices, including smartphones, tablets and desktop computers.

Progressive Web Applications provide features such as offline access, fast loading, home-screen installation, push notifications and background synchronization. They improve user experience by making web applications more reliable, responsive and accessible.

Examples of websites that have adopted PWA technologies include online shopping platforms, news websites, educational platforms and social networking applications.

2. Objectives of Progressive Web Applications

The main objectives of Progressive Web Applications are:

1. To provide an application-like experience through web browsers.
2. To make websites faster and more reliable.
3. To allow users to access websites even with limited internet connectivity.
4. To support installation on mobile and desktop devices.
5. To provide push notifications and background updates.
6. To improve website performance and user engagement.
7. To create applications that work across different platforms.

3. What is a Progressive Web Application?

A Progressive Web Application is a web application that uses modern browser technologies to provide features similar to native mobile applications.

Unlike traditional websites, PWAs can provide offline functionality, installation support and push notifications.

A PWA is generally built using three main technologies:

- HTML: Defines the structure and content of the application.
- CSS: Controls the design, layout and appearance.
- JavaScript: Provides interactive functionality and application logic.

PWAs also use a web app manifest and service worker to provide advanced application features.

4. Important Features of PWA

Progressive Web Applications provide several important features.

1. Responsive Design: PWAs automatically adjust their layout according to the screen size of mobile phones, tablets and desktops.

2. Offline Support: PWAs can store important files and information using caching techniques. This allows users to access previously available content without an internet connection.

3. Installability: Users can install a PWA on their device and access it through a home-screen icon.

4. Fast Loading: Cached resources help applications load quickly and reduce repeated network requests.

5. Push Notifications: PWAs can send notifications to users about new messages, offers, updates and important information.

6. Security: PWAs are generally served through HTTPS, which protects communication between the browser and server.

7. Cross-Platform Compatibility: A single PWA can work across supported mobile and desktop browsers.

5. Architecture of Progressive Web Applications

PWA architecture consists of different components that work together to provide a reliable and interactive application.

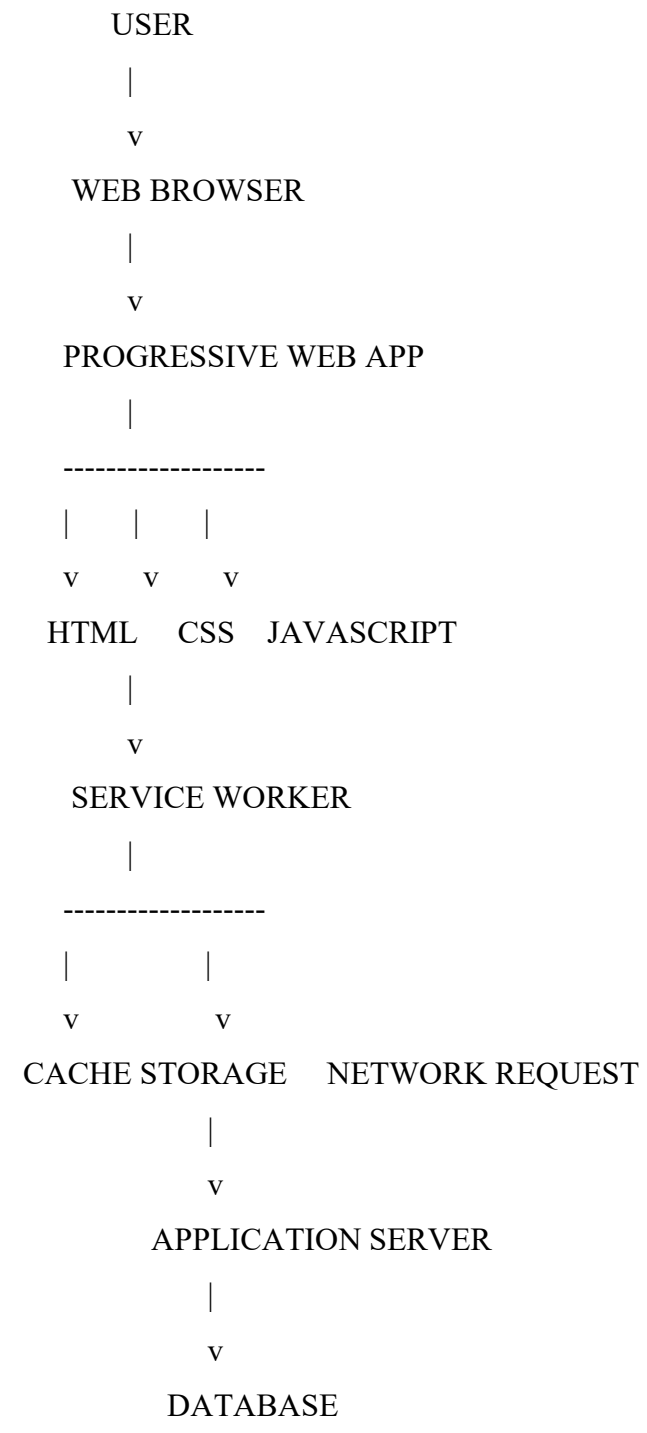
The main components are:

1. User Interface
2. Web Application
3. Web App Manifest
4. Service Worker
5. Cache Storage
6. Application Server
7. Push Notification Service

Architecture explanation:

- The user accesses the PWA through a web browser.
- The browser loads the HTML, CSS and JavaScript files.
- The web app manifest provides installation-related information.
- The service worker operates in the background and handles network requests.
- Cache Storage stores selected resources for offline access.
- The application server provides data and services.
- The push notification service delivers notifications to users when supported and configured.

Basic architecture diagram:



USER

6. Working Principles of PWA

Progressive Web Applications work through a combination of browser technologies and background services.

The working process involves the following steps:

1. The user opens the PWA through a web browser.
2. The browser downloads the application files from the web server.
3. The browser registers the service worker.
4. The service worker installs and stores selected files in the cache.
5. When the user opens the application again, cached resources can be loaded quickly.
6. If the internet connection is unavailable, the service worker can provide previously cached content.
7. The web app manifest allows users to install the application on supported devices.
8. Push notifications can be delivered through a configured push service.

Thus, PWAs provide a combination of web accessibility and application-like functionality.

7. Service Workers in PWA

A service worker is a JavaScript file that runs separately from the main web page in the browser.

It acts as an intermediate layer between the web application, network and cache storage.

Service workers are responsible for important PWA functions such as offline support, caching, background synchronization and push notifications.

Main functions of service workers:

- Intercept network requests.
- Store important resources in cache storage.
- Provide offline access.
- Load cached content when the internet is unavailable.
- Support background synchronization.
- Handle push notification events.

A service worker is registered using JavaScript.

Example:

```
if ("serviceWorker" in navigator) {
  navigator.serviceWorker.register("sw.js")
    .then(() => console.log("Service Worker Registered"))
    .catch(error => console.log(error));
}
```

This code checks whether the browser supports service workers and registers the sw.js file.

Service workers require a secure context, generally HTTPS or localhost during development.

8. Lifecycle of a Service Worker

A service worker has different lifecycle stages.

1. Registration: The browser registers the service worker file using JavaScript.

2. Installation: The service worker installs and can store important application files in the cache.

3. Activation: The service worker becomes active and can manage requests from its controlled pages.

4. Fetch: It intercepts network requests and decides whether to return cached resources or fetch them from the network.

5. Update: When a new version of the service worker is available, the browser installs and activates the updated version according to its lifecycle rules.

Example:

```
self.addEventListener("install", event => {
   console.log("Service Worker Installed");
});

self.addEventListener("activate", event => {
   console.log("Service Worker Activated");
});

self.addEventListener("fetch", event => {
   console.log("Fetching Resource");
});
```

These events help developers manage the different stages of a service worker.

9. Web App Manifest

A web app manifest is a JSON file that provides information about a Progressive Web Application.

It tells the browser how the application should appear when installed on a device.

The manifest contains information such as:

- Application name
- Short name
- Application icons
- Start URL
- Display mode
- Theme colour
- Background colour

The manifest is linked to the HTML page using the link element.

HTML example:

```
<link rel="manifest" href="manifest.json">
```

Manifest file example:

```
{
  "name": "My PWA",
  "short_name": "PWA",
  "start_url": "index.html",
  "display": "standalone",
  "background_color": "#ffffff",
  "theme_color": "#3498db",
  "icons": [
    {
      "src": "icon.png",
      "sizes": "192x192",
      "type": "image/png"
    }
  ]
}
```

This manifest provides the browser with the basic information required to present the application during installation.

10. Caching Strategies in PWA

Caching is an important feature of Progressive Web Applications. It stores frequently used files and resources so that they can be accessed quickly without downloading them repeatedly. Caching improves performance and supports offline functionality.

The major caching strategies are:

1. Cache First:

In this strategy, the service worker checks the cache first. If the requested resource is available, it is returned from the cache. Otherwise, the browser fetches it from the network.

It is useful for static resources such as images, CSS and JavaScript files.

2. Network First:

In this strategy, the service worker tries to obtain the latest resource from the network first. If the network is unavailable, it uses the cached version.

It is useful for frequently updated information such as news and user data.

3. Stale While Revalidate:

This strategy immediately returns the cached resource while requesting an updated version from the network in the background.

It provides faster loading while keeping cached content updated.

4. Cache Only:

This strategy returns resources only from the cache. It does not make a network request.

It is suitable for resources that are already stored and do not need to change.

5. Network Only:

This strategy always requests resources from the network and does not use the cache.

It is suitable for operations that require fresh server information.

Strategy	**Working principle**	**Application**
Cache First	Cache is checked first	Images and static files
Network First	Network is checked first	Latest news and data
Stale While Revalidate	Cache is returned while updating	Frequently accessed content
Cache Only	Only cached resources are used	Offline resources
Network Only	Only network resources are used	Live information

11. Offline Functionality in PWA

Offline functionality allows users to access certain application features even without an active internet connection.

This is achieved using service workers and Cache Storage.

During the installation stage, important resources such as HTML, CSS and JavaScript files can be stored in the cache.

When the user opens the application without an internet connection, the service worker checks the cache and returns the stored files.

Example:

```
const CACHE_NAME = "my-cache";


self.addEventListener("install", event => {
   event.waitUntil(
      caches.open(CACHE_NAME).then(cache => {
         return cache.addAll([
            "index.html",
            "style.css"
         ]);
      })
   );
});
```

This code creates a cache and stores the HTML and CSS files.

Offline functionality is useful for educational applications, shopping websites and travel applications.

12. Push Notifications in PWA

Push notifications are messages delivered to users even when they are not actively viewing the website.

They help websites communicate important updates and information.

For example, an online shopping application can send notifications about order status, discounts and new products.

Main components of push notifications:

1. User Permission: The application requests permission from the user to send notifications.
2. Push Subscription: The browser creates a subscription for receiving push messages.
3. Push Service: The push service delivers messages to the browser.
4. Service Worker: The service worker receives the push event.
5. Notification Display: The notification is displayed to the user.

Example:

```
Notification.requestPermission()
  .then(permission => {
    if (permission === "granted") {
      new Notification("Welcome!", {
        body: "You have a new update."
      });
    }
  });
```

This is a simple example of displaying a notification after permission is granted. Real remote push notifications also require a push subscription, a server and a push service.

13. Application Installation and User Experience

One of the important features of PWAs is their ability to be installed on supported devices.

Users can install a PWA through the browser's installation option without downloading it from a traditional app store.

After installation, the application can appear on the device's home screen with its own icon and launch experience.

The display property in the web app manifest controls how the installed application appears.

Common display modes include:

- standalone – opens like an independent application.

- fullscreen – uses the entire screen.
- minimal-ui – displays a limited set of browser controls.
- browser – opens in a normal browser window.

These features make PWAs convenient and easy to access.

14. Case Study: Online Shopping Progressive Web Application

An online shopping company wants to develop a Progressive Web Application to improve the shopping experience of its customers.

The application allows users to browse products, view product details, receive notifications and access previously loaded pages even when the internet connection is unavailable.

The application uses HTML, CSS and JavaScript for its interface and functionality.

Main features:

1. Product browsing and searching.
2. Responsive interface for mobile and desktop.
3. Offline access to previously cached product pages.
4. Fast loading through caching strategies.
5. Installation on supported devices.
6. Push notifications for offers and order updates.
7. Secure communication using HTTPS.

Working process:

- The customer opens the shopping website.
- The browser loads the application files.
- The service worker caches important resources.
- The user browses available products.
- Previously cached pages remain accessible during network interruptions.
- The web app manifest supports installation.
- Push notifications inform users about order updates and special offers.

This case study demonstrates how PWA technologies can improve accessibility, performance and customer engagement.

15. Advantages of Progressive Web Applications

Progressive Web Applications provide several advantages over traditional websites.

1. Fast Performance: Caching helps reduce loading time.
2. Offline Access: Previously cached content can be accessed without internet connectivity.
3. Easy Installation: Users can install the application directly through supported browsers.
4. Cross-Platform Support: PWAs can work on different operating systems and devices.
5. Lower Development Cost: A single web application can serve multiple platforms.

6. Push Notifications: Users can receive important updates.
7. Responsive Design: The application adjusts to different screen sizes.
8. Improved User Experience: App-like features make websites more convenient.

16. Limitations of Progressive Web Applications

Although PWAs provide many benefits, they also have some limitations.

1. Some advanced device features may not be supported equally across all browsers.
2. Push notification support varies depending on the browser and operating system.
3. Offline functionality requires proper caching implementation.
4. PWAs generally require HTTPS for secure operation.
5. Some users may not know that a website can be installed.
6. Certain native application capabilities may not be available or may require additional platform-specific implementation.
7. Developers must carefully manage cached data and application updates.

17. Testing and Evaluation of PWA

Testing is necessary to ensure that the Progressive Web Application works correctly.

The application should be tested for performance, offline access, installation, security and responsiveness.

Test	**Expected Result**
Application loading	Website loads successfully
Service worker	Service worker registers correctly
Manifest	Application information is displayed
Offline mode	Cached pages are available
Cache storage	Required resources are stored
Push notification	Notifications are delivered with permission
Installation	Application can be installed on supported devices
Responsive design	Layout adjusts to different screen sizes
HTTPS	Application uses a secure connection

Tools such as Google Chrome DevTools and Lighthouse can help developers test PWA features and identify problems.

18. Real-World Applications of PWA

Progressive Web Applications are useful in different industries.

1. E-commerce: Shopping applications use PWAs to provide faster browsing and order notifications.

2. Education: Educational platforms allow students to access previously loaded learning materials.

3. News and Media: News websites use caching and notifications to deliver updates.

4. Travel: Travel applications provide access to previously viewed booking information and travel details.

5. Banking: Banking websites can use PWA technologies to provide responsive interfaces and faster access to services, subject to appropriate security controls.

6. Social Networking: Social platforms can use notifications and app-like interfaces to improve communication.

19. Comparison Between Traditional Websites and PWA

Traditional Website	**Progressive Web Application**
Usually requires internet access	Can provide offline access
Generally accessed through a browser	Can also be installed
May load resources repeatedly	Uses caching for faster loading
Limited background functionality	Supports service worker features
Notifications may be limited	Can support push notifications
Browser-based experience	App-like user experience

20. Conclusion

Progressive Web Applications are an important development in modern web technology. They combine the accessibility of websites with several useful features of native mobile applications. The architecture of a PWA includes service workers, web app manifests, caching mechanisms, application servers and push notification services. These components work together to provide offline access, faster loading, installation support and improved user experience.

The online shopping case study demonstrates how PWA technologies can be used in real-world applications. Overall, Progressive Web Applications provide a cost-effective, reliable and user-friendly approach to modern web development.

QUESTION 5: CODE :

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>My Progressive Web App</title>

  <link rel="manifest" href="manifest.json">

  <meta name="theme-color" content="#3498db">

  <style>
    * {
      box-sizing: border-box;
    }

    body {
      font-family: Arial, sans-serif;
      margin: 0;
      background: #f0f4f8;
      text-align: center;
      color: #333;
    }

    header {
      background: #3498db;
      color: white;
      padding: 30px;
    }

    .container {
      padding: 30px 15px;
    }
```

```
        .card {
            background: white;
            padding: 25px;
            margin: 20px auto;
            max-width: 450px;
            border-radius: 12px;
            box-shadow: 0 3px 10px #ccc;
        }

        button {
            background: #3498db;
            color: white;
            border: none;
            padding: 12px 20px;
            margin: 8px;
            border-radius: 5px;
            cursor: pointer;
        }

        button:hover {
            background: #217dbb;
        }

        footer {
            background: #222;
            color: white;
            padding: 15px;
            margin-top: 30px;
        }
    </style>
</head>

<body>

    <header>
```

```
    <h1>My Progressive Web App</h1>
    <p>Fast, Reliable and Installable Website</p>
</header>

<div class="container">

    <div class="card">
        <h2>Welcome!</h2>
        <p>
            This application demonstrates
            Progressive Web Application features.
        </p>

        <button onclick="showNotification()">
            Show Notification
        </button>

        <button id="installBtn" hidden>
            Install App
        </button>
    </div>

    <div class="card">
        <h2>PWA Features</h2>
        <p>Offline Support</p>
        <p>Fast Loading</p>
        <p>Application Installation</p>
        <p>Service Worker</p>
        <p>Cache Storage</p>
    </div>

    <div class="card">
        <h2>Connection Status</h2>
        <p id="status">Checking connection...</p>
    </div>
```

```
</div>

<footer>
  <p>&copy; 2026 My Progressive Web App</p>
</footer>

<script>
  // Register Service Worker
  if ("serviceWorker" in navigator) {
    navigator.serviceWorker.register("sw.js")
      .then(() => {
        console.log("Service Worker Registered");
      })
      .catch(error => {
        console.log("Registration failed:", error);
      });
  }

  // Internet connection status
  function updateStatus() {
    document.getElementById("status").innerHTML =
      navigator.onLine ? "Online" : "Offline";
  }

  window.addEventListener("online", updateStatus);
  window.addEventListener("offline", updateStatus);

  updateStatus();

  // Notification demonstration
  function showNotification() {
    if ("Notification" in window) {
      Notification.requestPermission().then(permission => {
        if (permission === "granted") {
          new Notification("Welcome!", {
            body: "You are using a Progressive Web App."
```

```
                });
            } else {
                alert("Notification permission was not granted.");
            }
        });
    } else {
        alert("Notifications are not supported.");
    }
}

// Install PWA
let installPrompt;

window.addEventListener("beforeinstallprompt", event => {
    event.preventDefault();
    installPrompt = event;
    document.getElementById("installBtn").hidden = false;
});

document.getElementById("installBtn").addEventListener("click", async () => {
    if (!installPrompt) return;

    installPrompt.prompt();
    await installPrompt.userChoice;
    installPrompt = null;

    document.getElementById("installBtn").hidden = true;
});
</script>

</body>
</html>
```

QUESTION 5: OUTPUT:

